Full Name: Reda Nadir

Student ID: 159229

Course Code & Section: Session 1 - Group D

GitHub Username: xizvk

1. Repository Setup

I started by initializing the local folder as a Git repository using `git init`. For the `.gitignore` configuration, I set rules to exclude local environment configuration files (`local.env`) and generic log files (`*.log`), while explicitly adding a negative exception rule (`!important.log`) to ensure `important.log` remained tracked and visible to Git.

2. README & Commit History

The `README.md` file contains my full name, GitHub username, a brief description of the project, the technologies used, a sample code block, a markdown link, and the project banner image (`assets/project-banner.svg`). I organized my commit history into logical steps across multiple files, using conventional commit prefixes (like `chore`, `docs`, and `feat`) to keep the history clean and easy to follow.

3. Selective Staging

To demonstrate selective staging, I made modifications to two tracked files at the same time (`app.js` and `index.html`). Instead of staging everything, I explicitly ran `git add app.js` to stage only the first file, and then committed it using `git commit`. This left `index.html` modified in my working directory, which I verified using `git status`.

4. Branch & Merge

I created and switched to a dedicated feature branch named `feature/about` using `git checkout -b feature/about`. After adding the About section content and committing it on that branch, I switched back to the `main` branch via `git checkout main` and integrated the feature branch into `main` using `git merge feature/about`.

5. Remote Changes

A **commit** saves a snapshot of your changes locally on your machine, whereas a **push** uploads your local commits to the remote repository on GitHub. A **fetch** downloads

updates from the remote repository to your local machine without altering or merging them into your working files, while a **pull** performs a fetch and immediately merges those remote changes into your active branch.

6. Fair Use of AI

During this assignment, I used AI as a technical reference to double-check command syntax for selective staging workflows and to troubleshoot minor remote synchronization edge cases. I asked specific questions about how negative exceptions in `.gitignore` interact with wildcard rules. I reviewed the suggested solutions, tested the commands locally in my workflow, and adapted them to fit my repository structure before applying them.

7. Sources & Reflection

- **Sources:** Official Git documentation and course lab instructions.

- **Reflection:** The hardest concept to grasp initially was understanding the exact state offset between `fetch` and `pull`, particularly how local tracking branches lag behind remote references until merged. If Git behaves unexpectedly, the very first thing I check is `git status` to see the current state of the working tree and staging area.